Operating Systems Course - Laboratory Project

2026-04-27

1 Instructions

Please read the following instructions before starting the project.

- **Structure**. This project is worth 10 points out of the total grade of the course. As anticipated in the syllabus:
  - 20% of the grade will be based on the code you write: correctness, adherence to the specifications, code quality, etc.
  - 10% of the grade will be based on the report you write: clarity, completeness, presentation, etc.
  - 70% of the grade will be based on the presentation you will give at the exam.
- **Submission.** You **must** submit an archive that:
  - is named
    - `${surname1}_${surname2}_${surname3}.tar.gz` after the surnames of the

    group members. For example, if the surnames of the group members are Rossi, Bianchi and Verdi, the file must be named `Rossi_Bianchi_Verdi.tar.gz`. The order of the surnames does not matter.
  - contains a report, called `report.pdf`, which contains a description of your project, the design choices you made, and any other information you think is relevant. The report must be in PDF format and must be no longer than 5 pages.
  - contains a folder called `code`, which contains the source code of your project, organized as you wish. The code must be written in C and Bash, and it must compile and run correctly on the laboratory computers (or any Ubuntu 24.04 machine).
- **Compilation.** The project must include a way to compile the code and produce the necessary executables to run the project. You can choose to use either:
  - A `Makefile`, which compiles the code and produces the necessary executables to run the project. The Makefile must support at least the following targets:
    - `build`: compiles the project and produces the files required to run the project (check below for the required executables).
    - `clean`: removes all compiled files and executables, and restores the system to a clean state. It must also take care of cleaning up any IPC resources used by the project (e.g., named pipes).
    - `run`: executes one (possibly out of many) scenario of the project, at your choice, using the program(s) that the project requires (e.g., the bootstrapping program, the user script, etc.). Tip: you can choose to use the `ARGS` variable to allow passing arguments to the bootstrapping program when running it (`make run ARGS "--num-cooks=5"`).
  - A Bash script, called `build.sh`, that does exactly the same things stated abpove for the Makefile, but using Bash instead of Make. The script must support at least the following commands:
    - `./build.sh build`: compiles the code and produces the necessary executables to run the project.
    - `./build.sh clean`: removes any compiled files and executables, and restores the system to a clean state.
    - `./build.sh run`: executes a scenario of the project, at your choice, using the program(s) that the project requires.

- **Limitations.**
  - You can freely work on the project with your group and ask for help from the instructors or online, but you must write the code yourself. Plagiarism will be verified using software tools and will be punished according to the university regulations.
  - Any project not written in C and Bash (even C++, even if it compiles and runs correctly) will not be accepted.
  - You can use third-party libraries so long as they are included in your submission and they cause no issues with compilation and execution.
  - Solutions that deviate from the specifications will be penalized depending on the severity of the deviation. If you are unsure about whether a certain design choice is acceptable, please ask the instructors (only after having read the specifications again).
- **Deadline.**
  - You (as a group) can choose to submit the project at any exam attempt of your choice: June, July, September (2026); January, or February (2027). Coordinating with your group members to choose the best time for you is your responsibility.
  - The attempt in which you submit the project will be the attempt in which you will attend the exam, **as a group**. If you submit the project in the June attempt, you will attend the exam in June, and so on.
  - In the Project submission page you can see on Moodle, you will see a deadline. That deadline is a hard deadline for the next exam attempt and is no later than five days before the exam date you see on Esse3. Late submissions will not be possible, as Moodle will automatically close the submission page at the deadline.
  - Once the attempt has passed, the instructors will re-open the submission page for the next attempt, and so on.
  - You can upload, delete and re-upload the project files as many times as you want. Once you are sure that your project is ready for submission, press the "Submit" button in Moodle to submit the project. That action will lock the submission page and make your submission final for that attempt.

2 Project 2026-1 - *Library*

2.1 Setting

This project is set in a library system, like, e.g., the Sistema Bibliotecario Trentino. It involves multiple libraries (processes) that can communicate with each other to move books between each other and satisfy user requests to borrow books. Each library has a catalog of available books, which is bootstrapped at start. Libraries contact each other using IPC mechanisms we have seen in class.

2.2 Specification

2.2.1 Book

A book is a simple data structure that contains the following fields:

- Title: a string that represents the title of the book.
- Author: a string that represents the author of the book.
- Year: an integer that represents the year of publication of the book.

Feel free to use any data structure you like to represent a book in your code, as long as it contains the above fields. You can also add more fields if you want.

For simplicity, each book exists exactly once in the system, i.e., there are no multiple copies of the same book. This means that if a book is borrowed from one library, it is not available in any other library until it is returned.

2.2.2 User

A User is identified by a unique username (string). Users can perform the following operations:

- register with a library
- search for a book in a library
- borrow a book from a library
- return a book to a library

All these operations are carried out with a Bash script, called `user.sh`, that interacts with any library process (see below) to perform the operations.

No authentication or authorization is required for users: the script simply provides the username as a parameter to the library when performing operations. Users must first register with a library before they can borrow books. Users can register with as many libraries as they want, but they can only borrow books from the libraries they are registered with.

A user can only borrow one book at a time, and they must return the book before they can borrow another one. For simplicity, no due dates or fines will be implemented in this project.

Users can also request to borrow a book that is not available in their library, in which case the library will try to find the book in other libraries and borrow it from there if it is available. This process must be transparent to the user, who will simply receive a response indicating whether the book is available or not.

2.2.3 Library

A library is a C process that maintains a catalog of available books and supports two operations: library search and book borrowing/returning.

Each library has a globally unique ID (string or integer) that identifies it in the system. Libraries can communicate with each other using any IPC mechanism we have seen in class (pipes, FIFOs, message queues, sockets). The choice of IPC mechanism is up to you, but you must justify your choice in the report.

A library can search for a book in its catalog and return whether it is available or not. Search can be performed by title, author, or year. If the book is not found in the library's own catalog, the library can query other libraries and add the results to the response.

Users can request to borrow a book from a library. If the book is not available in the library's own catalog, the library must query and borrow the book from other libraries if it is available there. All cases must be handled, such as:

- Book available: the library can lend the book to the user and mark it as unavailable.
- Book lent to someone else (either in the library or in another library)
- Book not found in own catalog, but available elsewhere
- Book found in no catalog at all
- User tries to borrow a book while they already have one borrowed
- User tries to borrow a book without registering first
- User tries to return a book that they have not borrowed, or that they have already returned
- User tries to return a book to a library that is not the one they borrowed it from
- and any other edge case you can think of.

2.2.4 Book catalog

In the project files, you will find a file called `books.csv`. When bootstrapping the system, you **must** read this file (using Bash), split it into N catalogs (N = number of libraries), and save each catalog in a separate file (e.g., `catalog1.csv`, `catalog2.csv`, etc.). Each library will read its own catalog file at startup and use it to initialize its catalog of available books.

Once the system is running, the catalog is assumed to be static (i.e., no new books will be added or removed), so you do not need to implement any functionality to add or remove books from the catalog. However, you can implement such functionality if you want, as long as it does not interfere with the main functionalities of the system.

Books belong to their original catalog. A user must always return the book to the library they borrowed it from, even if it was fetched from another one. Then, it will be the borrowing library's responsibility to return the book to the library it belongs to, if necessary.

2.2.5 Library system

Libraries must implement a communication protocol to allow them to communicate with each other and satisfy user requests. The communication protocol can be implemented using any IPC mechanism we have seen in class (pipes, FIFOs, shared memory, sockets). The choice of IPC mechanism is up to you, but you must justify your choice in the report.

How you implement the communication protocol is also up to you, but it must allow libraries to:

- Send and receive requests to borrow books from other libraries.
- Send responses to requests, including the success or failure of the request and any relevant information (e.g., the location of the book if it is found in another library).
- **Handle concurrent requests** from multiple users and other libraries, ensuring that the system remains consistent and that books are not double-lent.
- **Handle errors** gracefully, such as when a book is not found or when a user tries to borrow a book that is already lent out.

2.2.6 Concurrency

- User requests can be either handled serially (by using synchronization mechanisms to ensure that only one request is processed at a time) or concurrently (by using threads to allow multiple requests to be processed at the same time). The choice of concurrency mechanism is up to you, but you must justify your choice in the report and ensure that the system remains consistent and that books are not double-lent.
- To allow for checking and assessing concurrency, libraries must wait a random amount of time (between 1 and 5 seconds) before responding to any request, to simulate the time it takes to process the request.

2.2.7 Management script

A Bash script must be written to manage the library system. The script must perform the following tasks:

- Check how many libraries (processes) are running (communicating via signals)
- List the available books in each library and whether they are available or lent out
- List the registered users in each library, and for each user, the book they have borrowed (if any)
- Stop all library processes and clean up any IPC resources used by the system

The Bash script can use any method to communicate with the library processes (e.g., signals, IPC, etc.) to retrieve the necessary information and display it to the user, e.g.:

- signals + writing to a temporary file
- sending a request via IPC and waiting for the response
- etc.

2.2.8 Error handling

- All operations in the library system **must** use return codes to indicate the success or failure of the operation. The system **must** define a set of error codes that can be returned by the library operations, such as `BOOK_NOT_FOUND`, `BOOK_UNAVAILABLE`, `USER_NOT_FOUND`, etc. The error codes should be defined and used consistently by both C and Bash code, but do not need to be shared (e.g., you can `#define` the error code in C and use a variable in Bash, as long as they have the same value).
- Any parameter in any Bash and C function that can fail (e.g., due to invalid input, system errors, etc.) **must** handle the error gracefully.
- Any file operation (e.g., reading the book catalog, saving the state of the library, etc.) **must** check for errors and return appropriate errors if the operation fails (e.g., file not found).

2.2.9 Edge cases

Check what happens when:

- Two users try to borrow the same book at the same time from the same library.
- A user tries to borrow a book that has been lent to another user.
- User A queries Library 1 for a book that is not available, and Library 1 queries Library 2 for the book at the same time that User B queries Library 2 for the same book.
- A user tries to return a book that they have not borrowed, or that they have already returned.

The exact implementation on how a library contacts other libraries (e.g., DNS-like system, broadcasting, etc.) is up to you, but you must ensure that the system can handle the above edge cases correctly and consistently. Tip: introduce a time to live (TTL) for requests to prevent infinite loops in case of circular queries between libraries.

2.3 Interfaces

For the sake of testing and grading, the following interfaces **must** be implemented:

- Bootstrapping script

  ```
  ./bootstrap.sh <num_libraries> <source_csv>
  ```

- Library process

  ```
  ./library <library_id> <num_total_libraries> <catalog_file>
  ```

- User script

  ```
  ./user.sh <username> <library_id> <operation> [operation_args]
  ```

  Example operations:

  - `./user.sh Alice 1 register`
  - `./user.sh Alice 1 borrow "The Great Gatsby"`
  - `./user.sh Bob 2 return "The Great Gatsby"`
  - `./user.sh Charlie 1 search --by author "F. Scott Fitzgerald"`
  - `./user.sh Charlie 1 search --by title "1984"`
  - `./user.sh Charlie 1 search --by year 1984`

- Management script

  ```
  ./manage.sh <operation>
  ```

  Example operations:

- `./manage.sh status` - Triggers the signal/IPC to dump state and prints it
- `./manage.sh stop` - Cleanly shuts down all library processes and cleans up IPC
- `./manage.sh list_books` - Lists all books in all libraries and their availability
- `./manage.sh list_users` - Lists all users registered in all libraries and the books they have borrowed (if any)